

Sincronización de reporte mensual de finanzas

En lugar de generar un reporte mensual con los montos acumulados hasta la fecha actual cada que el titular lo pida explícitamente, se busca que esta información sea única para cada mes y, por lo tanto, permita la visualización constante y la recopilación de estadísticas financieras del titular.

Primero, se aplica un constraint requerido para una operación de UPSERT posterior.

```
ALTER TABLE reportes  
ADD CONSTRAINT reportes_titular_mes_anho_uq  
UNIQUE (titular_id, mes, anho);
```

Se crea un índice en la base de datos para optimizar la procedure encargada de encontrar todas las transacciones para un titular y rango de fechas dadas.

```
CREATE INDEX idx_transacciones_titular_fecha  
ON transacciones(titular_id, fecha_pago);
```

El procedure en cuestión se encargará del cálculo de los montos acumulados y balance neto del monto especificado.

```

CREATE OR REPLACE PROCEDURE generar_reporte_mensual(
    p_titular_id uuid,
    p_mes integer,
    p_anho integer
)
LANGUAGE plpgsql
AS $$ DECLARE
    v_ingresos          numeric(15, 2) := 0;
    v_gastos            numeric(15, 2) := 0;
    v_aportes           numeric(15, 2) := 0;
    v_balance           numeric(15, 2) := 0;
    v_fecha_inicio      date;
    v_fecha_fin         date;
BEGIN
    -- Calculate date range for the query to leverage the index
    v_fecha_inicio := make_date(p_anho, p_mes, 1);
    v_fecha_fin    := (v_fecha_inicio + INTERVAL '1 month')::date;

    -- Aggregate transactions for the specified period
    SELECT
        COALESCE(SUM(CASE WHEN tipo = 'INGRESO' THEN monto ELSE 0 END), 0),
        COALESCE(SUM(CASE WHEN tipo = 'GASTO' THEN monto ELSE 0 END), 0),
        COALESCE(SUM(CASE WHEN tipo = 'APORTE_META' THEN monto
                                WHEN tipo = 'RETIRO_META' THEN -monto
                                ELSE 0 END), 0)
    INTO
        v_ingresos,
        v_gastos,
        v_aportes
    FROM transacciones
    WHERE titular_id = p_titular_id
        AND fecha_pago >= v_fecha_inicio
        AND fecha_pago < v_fecha_fin;

    -- Calculate net balance
    v_balance := v_ingresos - v_gastos;

    -- Perform Upsert (Insert or Update)
    INSERT INTO reportes (
        titular_id,
        mes,
        anho,
        ingresos_acmds,
        gastos_acmds,
        aportes_ahorros_acmds,
        balance_neto,
        fecha_generado
    ) VALUES (
        p_titular_id,
        p_mes,
        p_anho,
        v_ingresos,
        v_gastos,
        v_aportes,
        v_balance,

```

```
        now()
    )
    ON CONFLICT (titular_id, mes, anho)
    DO UPDATE SET
        ingresos_acmds = EXCLUDED.ingresos_acmds,
        gastos_acmds = EXCLUDED.gastos_acmds,
        aportes_ahorros_acmds = EXCLUDED.aportes_ahorros_acmds,
        balance_neto = EXCLUDED.balance_neto,
        fecha_generado = now();

END;
$;$
```

Finalmente, el Trigger determina el titular_id y el mes/año del reporte cambiado y llama la procedure para recalcular los montos respectivos.

```

CREATE OR REPLACE FUNCTION trg_fn_actualizar_reporte_mensual()
RETURNS TRIGGER AS $$ BEGIN
    IF TG_OP = 'INSERT' THEN
        -- New transaction added: Update report for that month
        CALL generar_reporte_mensual(
            NEW.titular_id,
            EXTRACT(MONTH FROM NEW.fecha_pago)::integer,
            EXTRACT(YEAR FROM NEW.fecha_pago)::integer
        );

    ELSIF TG_OP = 'DELETE' THEN
        -- Transaction removed: Update report for that month
        CALL generar_reporte_mensual(
            OLD.titular_id,
            EXTRACT(MONTH FROM OLD.fecha_pago)::integer,
            EXTRACT(YEAR FROM OLD.fecha_pago)::integer
        );

    ELSIF TG_OP = 'UPDATE' THEN
        -- Check if the date moved to a different month
        IF OLD.fecha_pago IS DISTINCT FROM NEW.fecha_pago THEN

            -- 1. Recalculate the OLD period
            CALL generar_reporte_mensual(
                OLD.titular_id,
                EXTRACT(MONTH FROM OLD.fecha_pago)::integer,
                EXTRACT(YEAR FROM OLD.fecha_pago)::integer
            );
            -- 2. Recalculate the NEW period
            CALL generar_reporte_mensual(
                NEW.titular_id,
                EXTRACT(MONTH FROM NEW.fecha_pago)::integer,
                EXTRACT(YEAR FROM NEW.fecha_pago)::integer
            );

            -- Check if financial details changed within the same month
            ELSIF OLD.monto IS DISTINCT FROM NEW.monto
                OR OLD.tipo IS DISTINCT FROM NEW.tipo THEN

                CALL generar_reporte_mensual(
                    NEW.titular_id,
                    EXTRACT(MONTH FROM NEW.fecha_pago)::INTEGER,
                    EXTRACT(YEAR FROM NEW.fecha_pago)::INTEGER
                );
            END IF;
        END IF;

    RETURN NULL; -- Result is ignored for an AFTER trigger
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_transacciones_reporte ON transacciones;
CREATE TRIGGER trg_transacciones_reporte
AFTER INSERT OR UPDATE OF monto, tipo, fecha_pago OR DELETE

```

```
ON transacciones  
FOR EACH ROW  
EXECUTE FUNCTION trg_fn_actualizar_reporte_mensual();
```

Manejo de contribuciones a metas de ahorro

En cuanto a las metas de ahorro, se manejan contribuciones o retiros segun el tipo de transacción asignado. Nuestra decision de diseño es tal que "Cuando se crea una transaccion de tipo APOORTE_META o RETIRO_META, se debe de enlazar a metas_ahorro usando la tabla transacciones_metas y actualizar el avance de la correspondiente meta de ahorro."

Primero, para optimizar la busqueda de transacciones pertenecientes a una meta, se crea el siguientes indice.

```
CREATE INDEX IF NOT EXISTS idx_transacciones_metas_meta_id ON transacciones_metas(  
    meta_id);
```

Se procede a componer una funcion Trigger para calcule el avance y actualice el estado de una meta de ahorro.

```

CREATE OR REPLACE FUNCTION fn_actualizar_avance_meta()
RETURNS TRIGGER AS $$ DECLARE
    v_meta_id uuid;
    v_monto_objetivo double precision;
    v_fecha_limite date;
    v_total_actual double precision;
    v_nuevo_avance integer;
BEGIN
    -- Determine the meta_id based on the trigger event
    -- Case A: Trigger fired from transacciones_metas (Insert/Delete of link)
    IF TG_TABLE_NAME = 'transacciones_metas' THEN
        IF TG_OP = 'DELETE' THEN
            v_meta_id := OLD.meta_id;
        ELSE
            v_meta_id := NEW.meta_id;
        END IF;

    -- Case B: Trigger fired from transacciones (Update of amount/type)
    ELSIF TG_TABLE_NAME = 'transacciones' THEN
        IF (NEW.monto = OLD.monto AND NEW.tipo = OLD.tipo) THEN
            RETURN NULL;
        END IF;

        SELECT tm.meta_id INTO v_meta_id
        FROM transacciones_metas tm
        WHERE tm.transaccion_id = NEW.transaccion_id;

        -- If this transaction isn't linked to a goal, stop.
        IF v_meta_id IS NULL THEN
            RETURN NULL;
        END IF;
    END IF;

    SELECT monto_objetivo, fecha_limite
    INTO v_monto_objetivo, v_fecha_limite
    FROM metas_ahorro
    WHERE meta_id = v_meta_id;

    -- Calculate Total Accumulated Amount
    SELECT COALESCE(SUM(
        CASE t.tipo
            WHEN 'APORTE_META' THEN t.monto
            WHEN 'RETIRO_META' THEN -t.monto
            ELSE 0
        END
    ), 0)
    INTO v_total_actual
    FROM transacciones t
    JOIN transacciones_metas tm ON t.transaccion_id = tm.transaccion_id
    WHERE tm.meta_id = v_meta_id;

    -- Calculate New Progress Percentage
    v_nuevo_avance := (v_total_actual / v_monto_objetivo * 100)::integer;

    IF v_nuevo_avance < 0 THEN v_nuevo_avance := 0; END IF;

```

```

-- Update Goal Status and Avance
UPDATE metas_ahorro
SET
    avance = v_nuevo_avance,
    estado = CASE
        WHEN v_nuevo_avance >= 100 THEN 'COMPLETADA'
        WHEN CURRENT_DATE > v_fecha_limite THEN 'VENCIDA'
        ELSE 'EN_PROGRESO'
    END
WHERE meta_id = v_meta_id;

RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

Se crean dos triggers: uno para transacciones_metas y otro para transacciones.

El Trigger de transacciones maneja actualizaciones al monto o tipo de una transaccion pre existente.

```

CREATE TRIGGER tr_transacciones_actualizar_meta
AFTER UPDATE OF monto, tipo ON transacciones
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_avance_meta();

```

El Trigger de transacciones_metas maneja INSERT y/o DELETE de transacciones enlazadas a una meta.

```

CREATE TRIGGER tr_transacciones_metas_actualizar_meta
AFTER INSERT OR DELETE ON transacciones_metas
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_avance_meta();

```

Finalmente, una proceduro maneja la creacion y enlazamiento de las nuevas transaccion de tipo APORTE_META o RETIRO_META, hacia de transacciones_metas.

```

CREATE OR REPLACE PROCEDURE sp_registrar_transaccion_meta(
    p_titular_id uuid,
    p_nombre varchar(150),
    p_monto numeric(38, 2),
    p_fecha_pago date,
    p_tipo varchar(255),
    p_meta_id uuid,
    p_descripcion varchar(255) DEFAULT ''
)
LANGUAGE plpgsql
AS $$ DECLARE
    v_transaccion_id uuid;
BEGIN
    -- Validation: Ensure type is valid for a goal
    IF p_tipo NOT IN ('APORTE_META', 'RETIRO_META') THEN
        RAISE EXCEPTION 'El tipo de transacción debe ser APORTE_META o RETIRO_META';
    END IF;

    -- Validation: Ensure goal exists
    IF NOT EXISTS (SELECT 1 FROM metas_ahorro WHERE meta_id = p_meta_id) THEN
        RAISE EXCEPTION 'La meta de ahorro especificada no existe';
    END IF;

    -- Insert into transacciones
    INSERT INTO transacciones (
        titular_id,
        nombre,
        monto,
        fecha_pago,
        tipo,
        descripcion
    ) VALUES (
        p_titular_id,
        p_nombre,
        p_monto,
        p_fecha_pago,
        p_tipo,
        p_descripcion
    )
    RETURNING transaccion_id INTO v_transaccion_id;

    -- Insert into transacciones_metas
    INSERT INTO transacciones_metas (transaccion_id, meta_id)
    VALUES (v_transaccion_id, p_meta_id);
END;
    $$;

```